Exploration of Rewards Shaping and Imitation Learning in Chess

29.01.2026

Jakob Lambert-Hartmann

Contents

Introduction

Reinforcement learning (RL) is a field of machine learning concerned with decision-making. It is used in scenarios where an intelligent agent takes actions in an environment in order to maximize a reward (e.g. a chess agent learning to maximize the chance of wining). The actions chosen by the agent (e.g. chess move) are determined by a policy π. Whenever the agent takes an action, it receives a reward from the environment. Winning a game for instance will yield a positive reward, losing a negative reward, and otherwise the reward will be zero. Over the course of training the agent adapts its policy in order to maximize rewards (see Figure 1).

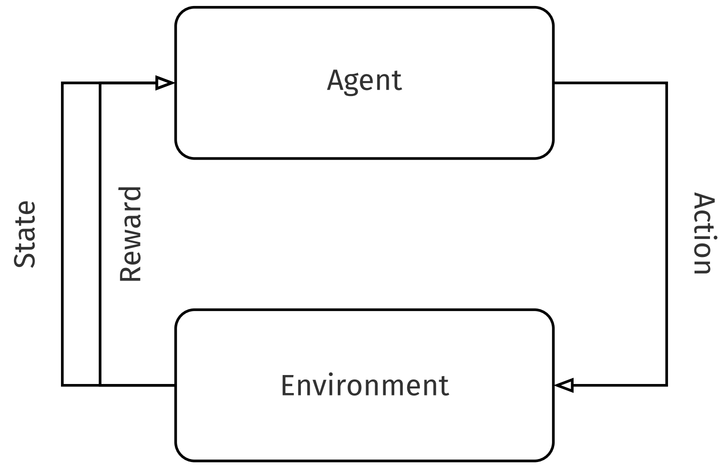

Figure 1: Reinforcement learning: Agent chooses action based on state. Environment reacts to the state and returns a new state and a reward to the agent.

If rewards are sparse, meaning most rewards are zero, it is difficult for an agent to learn a good policy, as there will be too little feedback for the model to learn. Wining a game of chess for instance is a sparse reward. It is difficult for an agent to learn chess with win as only reward, as it is very unlikely for an agent to randomly win a game. To help agents learn policies faster reward shaping introduces further rewards that align with the original goal and help to nudge the model in the right direction [1]. Reward shaping however is challenging and time-consuming. Common difficulties are:

Reward Hacking Agents exploiting unintended loopholes in the reward functions [2]

Misaligned Rewards Rewards are misaligned with the true objective [2]

Difficulty in Evaluation It is difficult to evaluate the effectiveness of a reward function [2]

Imitation learning, on the other hand, is a paradigm in reinforcement learning, in which an agent learns a policy through a set of expert demonstrations, rather than an explicit reward function. Behavior cloning is a specific imitation learning technique, which utilizes supervised learning to learn this policy. Using supervised learning circumvents reward shaping, however, expert demonstrations used for behavior cloning are often not uniformly sampled from the state space, which may lead to poor performance [3].

One Idea for mitigating the drawbacks of both reward based reinforcement learning and behavior cloning would be to combine the two approaches. Behavior cloning as a pretraining step could reduce the need for reward shaping, while fine-tuning with reward based reinforcement learning could iron out the sampling bias from the expert demonstrations. In this document we investigate the interaction between pretraining with behavior cloning and fine-tuning with reward shaping using different sets of expert demonstrations and reward functions, which we will apply to the game of chess.

Methods

To investigate the interaction between behavior cloning and reward shaping designed reward functions of varying densities, and selected expert demonstrations with varying sampling bias. We then trained models for all behavior cloning dataset and reward function combinations. For better generalizability of our findings, we performed the same experiment with three neural network architectures.

In the following sections we will describe our choices in model architecture, behavior cloning, and reward shaping in more detail.

Neural Network Architecture

We repeated our experiment using three neural network architectures: a fully connected neural network (see Figure 8), a convolutional neural network (see Figure 6) [4], and a residual network (see Figure 7) [5].

While the architectures differ in their hidden layers, they share the same input and output representation. The **input** is a one hot encoding of the current position. There are 8×8 squares on a chess board and 12 distinct pieces (black and white version of: pawn, rook, knight, bishop, queen, and king). One simplification we made, is that it's always white to play. To generate a move for black, the position first needs to be mirrored. The reason is that otherwise, we would either need to train two models, or that the model would need to learn the same patterns, once for white and once for black. As chess is a symmetric game, this simplification does not reduce the model's capability.

The network **output** represents a distribution over moves. It is implemented as two tensors (from which move, to which move), with which we compute joint probability distribution. In chess the number of legal moves depends on the position (see Figure 2), as such the move distribution is computed from the set of legal moves. This representation limits the number of parameters for faster training, however it is not without drawbacks:

Limiting Agent Expressiveness In chess, when a Pawn moves up the board it can promote to any piece (e.g. transform into a queen). In our output representation it is only possible to promote to a queen. However, promotions are rare, and almost all pawns are promoted to a queen. This simplification should not affect model performance much.

Inductive Bias The representation of the move distribution as a joint probability introduces inductive bias, that may misalign with chess. This approach implies that the choice of $\mathrm{Square}_{\mathrm{from}}$ and $\mathrm{Square}_{\mathrm{to}}$ are independent. This does not hold true for all positions (e.g. moving the queen to a square may lead to a checkmate, while other pieces would prolong the game).

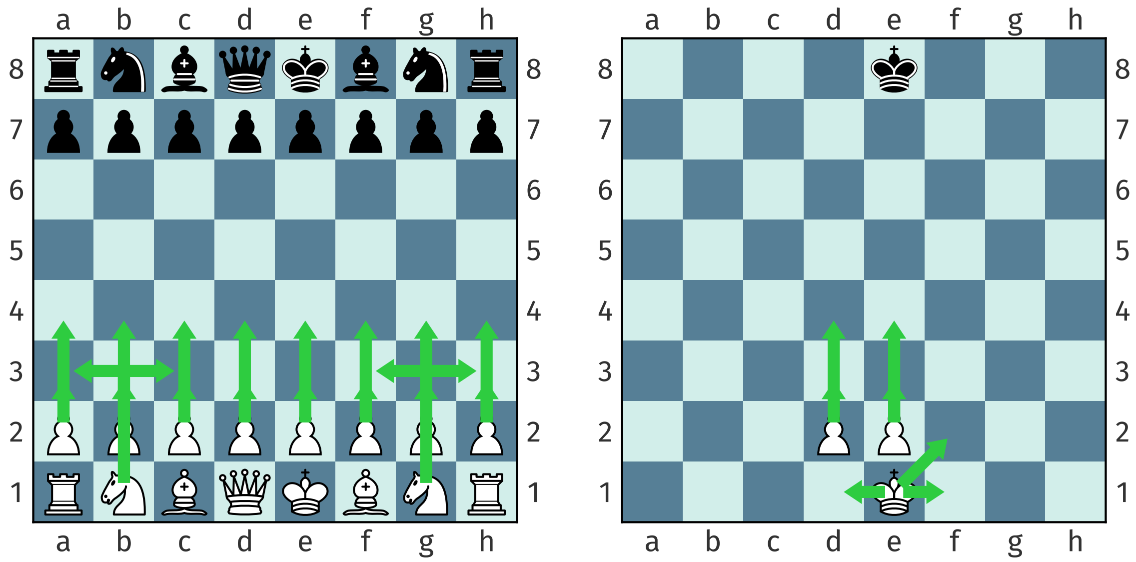

Figure 2: Red arrows show all the legal moves in both positions. As the number of legal moves changes depending on position, the model masks out illegal moves for to generate actions.

Behavior Cloning

For behavior cloning we chose two datasets (puzzles and master games) with different expert demonstration bias. Models were trained on either dataset for one or 10 epochs. To measure the effect of pretraining, we also had a model that was not subject to pretraining. The following sections will describe both datasets in further detail. Before describing the dataset bias however, we need to introduce some domain specific concepts.

Tactics and Strategy

For an agent to be successful in Chess it needs to master both strategies and tactics. **Tactics** defines moves that achieve short term gains. This may be capturing opponent pieces or checkmating the opponent king. **Strategy** on the other hand defines moves with a long term benefit. This could include positioning a piece to put pressure on the opponents king, which may lead to a checkmate in 20–30 moves. Figure 3 highlights those concepts with two example positions. One key difference between tactical moves and strategic moves, is the number of viable alternatives. Whereas tactics commonly has only one, or maybe two viable moves in a position, strategy often has a range of viable moves with little difference. This property affects how suitable both aspects are to supervised learning.

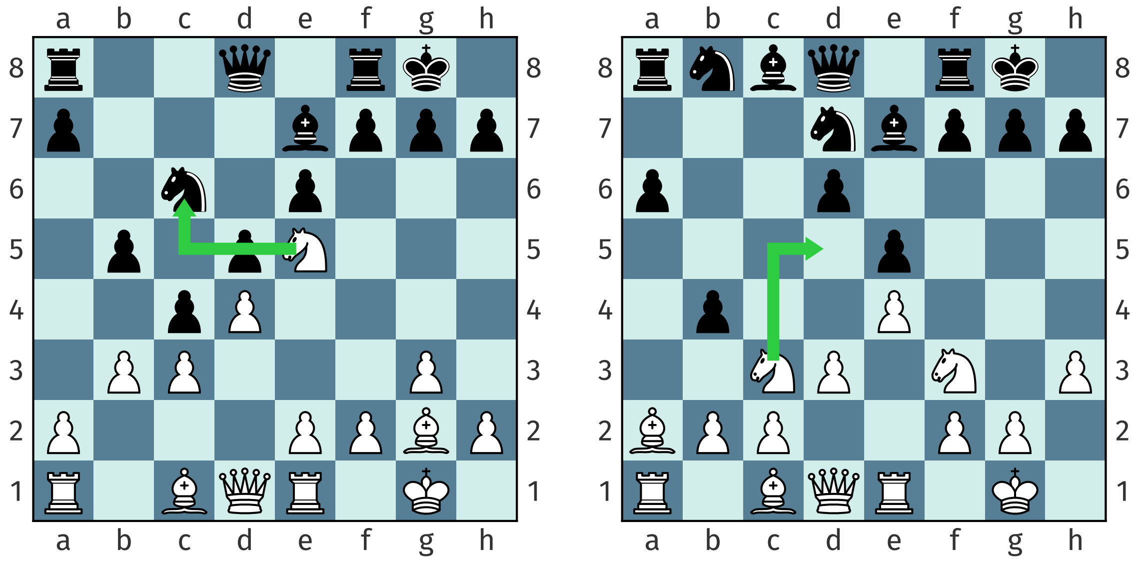

Figure 3: The position on the left has a tactical motif, where the black knight can be captured by white, as it is not defended. On the other hand the right position has a strategic motiv, where the knight is attacked and has to retreat. There are multiple available squares, but d5 is the best option. There is no immediate gain, however in the long run the knight will be able to exert more pressure on the opponent from this square.

Puzzle Data

Players that want to improve their tactical perception can train on puzzles. A chess puzzle contains a position with a tactical motif (e.g. checkmate, …). A player then needs to play 4–5 moves to prove their understanding of the motif.

There are many published datasets containing chess puzzles, such as the Lichess puzzle dataset [6]. Positions in these datasets are highly biased in that they solely contain tactical positions, underrepresenting strategic motifs. Furthermore, positions in such datasets always contain a tactical motif. The lack of negatives may lead the model to be tactically overconfident, leading to false positives (see Figure 4).

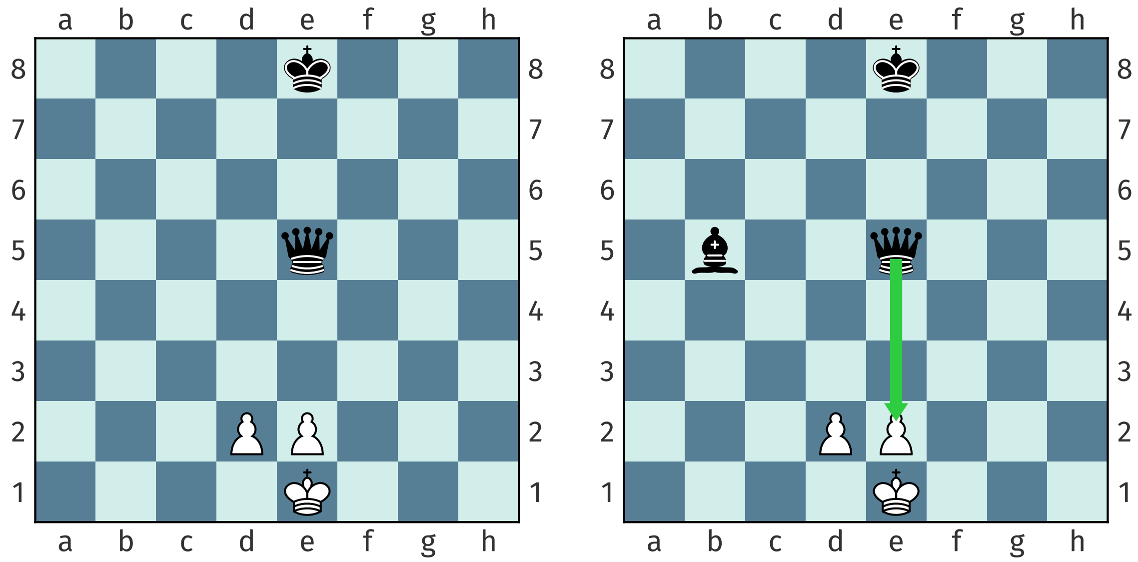

Figure 4: both positions are similar except for the black bishop on d5. In the position with the Bishop (right) black can win the game in one move by checkmating with the queen on e2. playing queen e2 in the other position will lose the game.

Grandmaster Data

Grandmaster games are overall less biased than the puzzle dataset. They cover entire games, and therefore contain positions with tactical and positions with strategic motifs. However, they still are biased in that they only represent top level play. Furthermore, Grandmasters are able to recognize a losing position earlier than club level players. As such, they often resign multiple moves before an unavoidable checkmate (see [7]). This underrepresentation of checkmates, may make it difficult for our model to convert a wining position into a checkmate (see Figure 5).

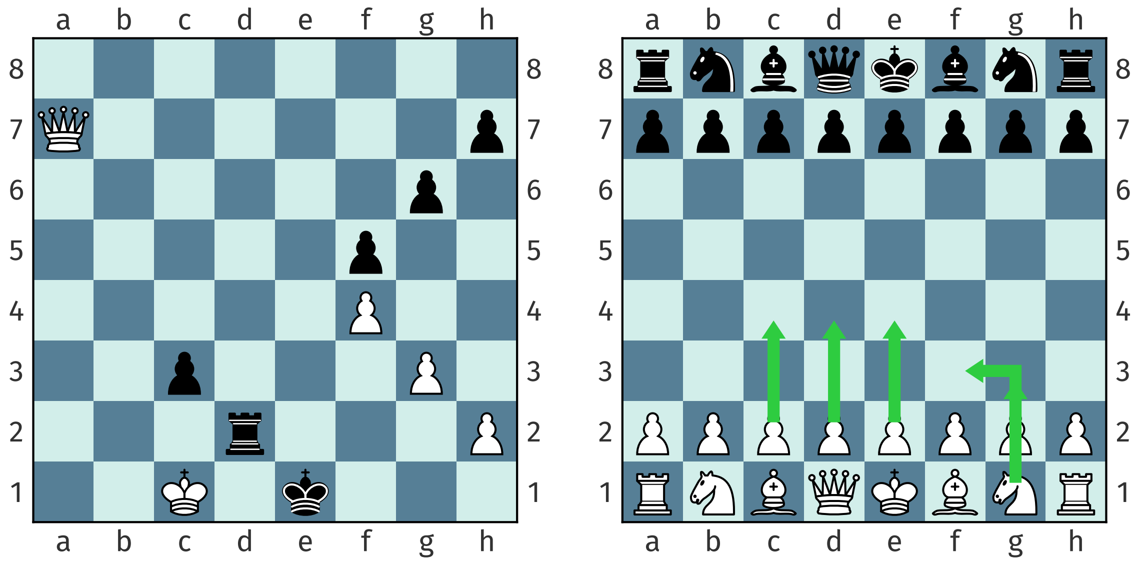

Figure 5: The position on the left shows the last position of a game by the two former world champions Garry Kasparov and Veselin Topalov. Black resigns in this completely lost position without waiting for a checkmate. The position on the right shows different first moves played by masters, showing that there is more than one viable move per position.

Rewards Shaping

Models trained with reinforcement learning through self play. The models played 16 games before updating based with REINFORCE [8]. This was performed for 1000 iterations, resulting in 16000 games played both as white and as black. After each chosen action a set of reward function was evaluated.

We used three sets of reward functions ranging form sparse to dense:

Sparse The sparse reward consisted of only one reward function, only a nonzero reward at the end of the game. A positive reward was awarded for a win, whereas a loss returned a negative reward.

Medium The reward set of medium sparsity returned material, control center, win, king safety

Dense The dense reward-set returned control outer center, control center, castling, promoting, blunder prevention, material, king safety, give check, win

In chess a win yields one point, a loss zero, and a draw half a point. To compare the models we summed points for each matchup (two models playing 500 games). We normalized the point values to the range $[0, 1]$, which represents the portion of available points the model received.

Results

In this chapter we will evaluate the trained models. For behavior cloning it is possible to evaluate the models on the test set. For models using reinforcement learning however, there is no test set to evaluate our model. Thus we evaluated the models though self play against each other. Each model played 500 games as white and 500 games as black against all model with the same architecture (i.e. CNN models only played CNN, etc.).

In chess there are three possible game outcomes: win, loss, and draw. To compute a single score we have used the tournament score system where a win is worth 1 point, a draw 0.5, and a loss 0.

We aggregated the points for each match (two models playing 1000 games). We then normalized the match results into a proportion of points. This value has a similar interpretation to a win percentage.

arch	cnn				fc				resnet			
rl	none	r_0	r_1	r_2	none	r_0	r_1	r_2	none	r_0	r_1	r_2
train												
none	nan	0.266	0.235	0.251	nan	0.289	0.307	0.335	nan	0.408	0.407	0.399
pz_1	0.400	0.478	0.564	0.433	0.507	0.697	0.631	0.623	0.830	0.410	0.410	0.409
pz_10	0.446	0.672	0.515	0.520	0.530	0.637	0.521	0.558	0.925	0.403	0.403	0.403
gm_1	0.396	0.605	0.650	0.572	0.390	0.483	0.523	0.494	0.797	0.411	0.409	0.424
gm_10	0.434	0.714	0.694	0.654	0.418	0.494	0.538	0.526	0.870	0.387	0.398	0.399

Table 1: Expected proportion of points for all models (best models highlighted in aqua). The proportions are means of all matchs (e.g. Model using residual network pretrained on puzzle data with no reinforcement learning, wins 92.5% of points against all other residual network models)

At first glance there is no clear superior training strategy (see Table 1). The best models, however, are either with sparse rewards or without reinforcement learning all together. In the following section we further investigate behavior cloning and reward shaping, and whether

Behavior Cloning

For models pretrained with supervised learning it is possible to evaluate them on their test sets. The models trained on puzzle data generalized much better on test (see Table 2). However, the test accuracy is probably not representative, as the Grandmaster dataset contains multiple data points where the same position has a different label.

dataset	epochs	fc	cnn	resnet
puzzle	1	62.0%	69.9%	74.6%
	10	65.8%	73.3%	81.7%
grand masters	1	31.0%	38.5%	41.5%
	10	31.8%	39.6%	43.7%

Table 2: Immitation learning training accuracy

The game results are a better metric to compare the performance of different pretraining strategy (see Table 3). Again, there is no clear pretraining strategy that dominates, however there is a significant improvement in pretraining of any length with any dataset compared to pure reinforcement learning.

dataset	epochs	fc	cnn	resnet
none		0.310	0.251	0.405
puzzle	1	0.615	0.469	0.515
	10	0.562	0.538	0.533
grand masters	1	0.472	0.556	0.510
	10	0.494	0.624	0.513

Table 3: Mean expected proportion of points by pretraining strategy (best model of architecture highlighted).

When only comparing models that did not use reinforcement learning however, pretraining on puzzle data for 10 epochs yields the best performance across all architectures (see Table 4).

dataset	epochs	fc	cnn	resnet
puzzle	1	0.507	0.400	0.830
	10	0.530	0.446	0.925
grand masters	1	0.390	0.396	0.797
	10	0.418	0.434	0.870

Table 4: Expected proportion of points for models not using reinforcement learning. (best model of architecture highlighted).

In conclusion, training on puzzle games for 10 epochs yielded the best results without any reinforcement learning. This pretraining strategy however, does retain its advantage when coupled with reinforcement learning.

Reward Shaping

When comparing the reward functions in models that were not pretrained, there is no clear reward function set, that dominates the others (see Table 5). The fully connected network increases performance as the rewards get denser. The CNN and ResNet however show no trend.

rewards	fc	cnn	resnet
none	0.461	0.419	0.855
r_0	0.520	0.547	0.404
r_1	0.504	0.532	0.405
r_2	0.507	0.486	0.407

rewards	fc	cnn	resnet
r_0	0.289	0.266	0.408
r_1	0.307	0.235	0.408
r_2	0.335	0.251	0.399

Table 5: Expected points by reward set. Left: across all models. Right: across models wihtout any supervised pretraining.

When considering the reward set across all pretraining strategy however, models with sparse rewards tend to perform slightly better. For the residual network however, there is a significant drop in performance after any type of reinforcement learning (see Table 5). The results highlight the complexity of reward shaping, as our rewards which should have guided the model towards better policies, did not show any increase in performance.

Conclusion

Out of the training strategies used, there was no strategy that performed better across all network architecture.

Mixing puzzles and GM games Training on a mix of puzzle and Grandmaster positions could reduce the demonstration bias.

Data preparation on GM games Multiple labels for the same data points prevented the models to generalize well. A strategy to unify the labels could improve the model's performance (e.g. selecting one move or having move distributions as label).

Longer reinforcement learning Only training our models for 1000 batches of reinforcement learning may be too little for any significant improvement.

Smaller reinforcement learning batches Aggregating the loss for 16 games may hinder the model in learning, as it reduces noise too much.

Bibliography

[1] Tim Miller, “Reward Shaping.” [Online]. Available: https://gibberblot.github.io/rl-notes/single-agent/reward-shaping.html

[2] S. Ibrahim, M. Mostafa, A. Jnadi, H. Salloum, and P. Osinenko, “Comprehensive overview of reward engineering and shaping in advancing reinforcement learning applications,” *IEEE Access*, 2024.

[3] “Imitation Learning, Stanford University.” [Online]. Available: https://web.stanford.edu/class/cs237b/pdfs/lecture/lecture_10111213.pdf

[4] K. O'shea and R. Nash, “An introduction to convolutional neural networks,” *arXiv preprint arXiv:1511.08458*, 2015.

[5] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.

[6] “Lichess chess puzzle dataset.” [Online]. Available: https://www.kaggle.com/datasets/tianmin/lichess-chess-puzzle-dataset

[7] “All GM Chess Games on Chess.com.” [Online]. Available: https://www.kaggle.com/datasets/dimitrioskourtikakis/gm-games-chesscom

[8] J. Zhang, J. Kim, B. O'Donoghue, and S. P. Boyd, “Sample Efficient Reinforcement Learning with REINFORCE,” *CoRR*, 2020, [Online]. Available: https://arxiv.org/abs/2010.11364

Appendix

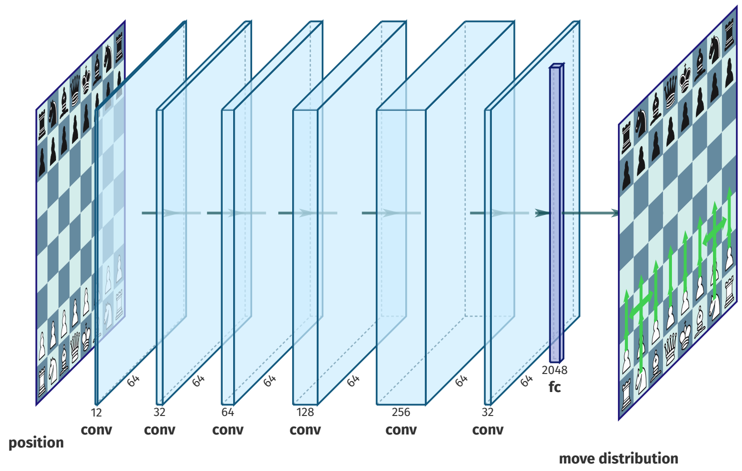

Figure 6: Convolutional neural network

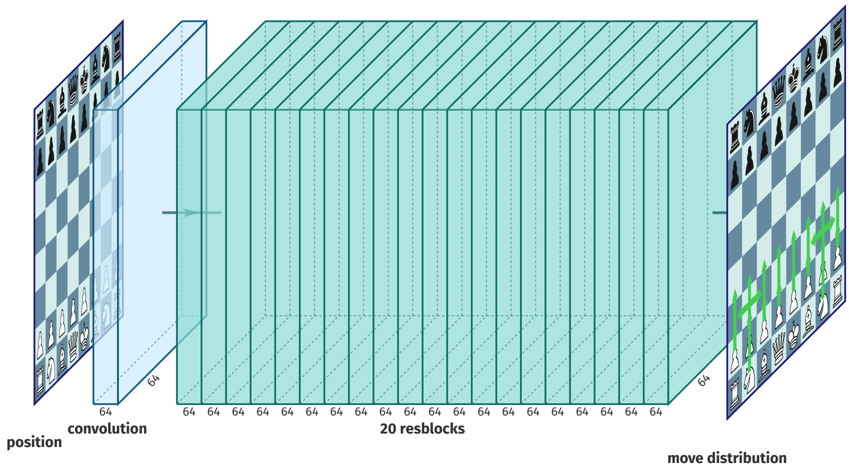

Figure 7: Residual block architecture

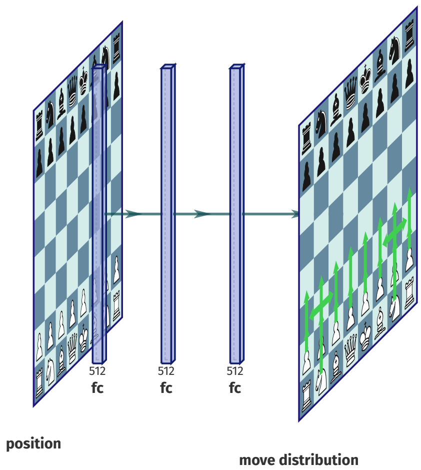

Figure 8: Fully connected network